

LSM-KV 项目报告

胡锦涛峰 522031910358

2024年 5月 28日

1 背景介绍

LSM树，即Log-Structured Merge Tree，日志结构合并树。其中的日志结构即指LSM树不对已写入的数据进行修改，而是进行追加，类似日志每次都新的记录追加到整个结构的末尾。而Merge即指将多条记录同时写入磁盘的过程。LSM树因为追加的机制使得可以做到对磁盘的顺序写入，因此会有更高的写性能，被广泛用于NoSql数据库的实现中。此次项目主要实现对原始LSMTree的键值分离优化，从而减少LSM树的写放大缺点，并完成垃圾回收等功能。

2 测试

2.1 性能测试

2.1.1 预期结果

常规分析中，因为Put只是单纯写入内存部分的Table，因此其平均延迟应该是最低的，并且在不同数据大小下变化幅度较小，不过因为Compaction的存在，平均延迟会随着数据的增大而增大。Get会从上层开始搜索，当数据量较大时，需要查看的SStable也更多，因此随着数据量增大，延迟应较显著增大。Delete需要先get再PUt，因此性能应该与get相近。Scan需要遍历所有的SStable,因此延迟应该是最大的，并且随着数据量增大，延迟显著增大。吞吐量与平均时延类似，不同的在于Put的吞吐量会因为Compaction的发生而被占用部分的时间，因此Put的吞吐量表现应该较时延来说差。

缓存测试中，缓存的数据越多，性能应该越好。

BF大小来说，BF越大则误报率越低，BF越小则一个sstable可以存放的key越多。同时，key的数量即BloomFilter中数据的个数，所以应该可以取到一个中间值，使得BF与Key数量恰好配合，使得在误报率较小的情况下尽可能地存放更多Key。

2.1.2 常规分析

1. 平均时延随KeyNum与值大小变化测试结果如下。随Key数量变化

KeyNum\TimeLatency(ns)	Put	Get	Delete	Scan
100000	2689.84	3597.12	9461.68	165001028.31
10000	2203.14	2678.99	5717.71	13169803.73
1000	1260.88	2467.50	5380.48	1611322.72
100	430	209.30	204.70	11916.75
10	560.15	195.26	210.30	17715.20

随值长度变化

KeyNum	ValLen\Latency(ns)	Put	Get	Delete	Scan
1000	100	1068.20	2408.54	4237.96	1446064.72
1000	1000	4088.83	2961.18	5182.02	1492788.04
1000	5000	8303.41	4043.35	7235.66	1614069.48
1000	10000	15736.16	5235.35	10387.71	1741358.95
100	100	592.88	212.65	206.59	11741.00
100	1000	899.10	280.43	206.412	14760.50
100	5000	1852.54	646.47	236.93	13382.30
100	10000	3034.96	810.66	243.52	10958.80

(a) 随KeyNum变化:

Put。可以看到随着KeyNum的增加, Put的平均时延略有上升, 原因是之前理论分析中提到的Compaction会被统计到总时间中, 因此导致平均时延的上升。

Get。Get时延随着keynum上升而上升, 因为每次Get时会在缓存中寻找, 所以会导致时延的上升。Delete因为是Get与Put结合, 因此同样上升。

Scan。Scan会随着KeyNum的变化而显著增大, 因为KeyNum上升时sstable数量增大, 需要扫描的sstable变多, 同时测试时的扫描范围也相应变大了。

(b) 随值长度变化:

随长度变化最显著的是Put操作, 因为Put操作会将读入的值写入到硬盘中, 写硬盘是一个昂贵操作, 且与写的大小成正比, 因此值长度越大, 消耗的时间越长。

另外Get,Delete也有一定程度的减慢, 是因为Get与Delete同样涉及读取硬盘值, 同样需要消耗一定的时间, 但总体来说读会比写消耗的时间更少, 因此变化没有那么显著。

Scan的性能略有减慢, 同理因为内存读操作, 但是Scan主要性能消耗在扫描sstable中, 读盘操作的占比较小, 所以性能减慢幅度不大。

2. 吞吐量变化 随KeyNum变化

KeyNum\throughput(per ms)	Put	Get	Delete	Scan
10	6566.27	6552.38	14567.33	947.13
100	5610.52	5686.97	11226.47	114.064
1000	756.43	369.61	4646.71	1.182
10000	216.65	281.67	2385.93	1.05
100000	203.18	201.24	85.68	1.002

随值大小变化

KeyNum	Vlen\throughput(per ms)	Put	Get	Delete	Scan
10	100	4653.04	5946.28	41581.84	959.45
10	1000	4416.523	4943.29	14376.43	943.63
10	10000	1537.81	2188.90	14970.57	958.11
10000	100	237.87	336.17	2660.24	1.08
10000	1000	144.84	326.69	2669.24	1.04
10000	10000	34.68	183.20	2517.45	1.04

在KeyNum变化中，Put的吞吐量与Get基本一致，Delete的吞吐量低于二者，Scan吞吐量急剧下降，与我们所估计的结果一致。

值大小变化中，选取两个具有一定代表性的Key数量观察规律，发现Put的吞吐量均随Vlen增大而快速下降，Get与Delete略微下降，这个与之前的分析，以及时延分析中大小变化是一致的。

2.1.3 索引缓存与Bloom Filter的效果测试

需要对比下面三种情况GET操作的平均时延，沿用之前常规分析时使用的测试程序进行Get时延测试。

1. 内存中没有缓存SSTable的任何信息，从磁盘中访问SSTable的索引，在找到offset之后读取数据 因为已经完成缓存与二分查找的操作，为了模拟该操作的性能同时不影响整个系统的正确与稳定性，在每次使用缓存读取时，打开SSTable文件并访问读取一次索引，使得Get操作的性能与磁盘访问索引的性能一致。具体模拟实现如图：

```
uint64_t midKey = ((uint64_t*)(cash+KEY_INDEX(mid,offset)))[0] ;
#ifdef CACHE_TEST1
    if(sstable)
    {
        char * buf = (char *)malloc(BUFFER_SIZE) ;
        rewind(sstable) ;
        fseek(sstable , KEY_INDEX(mid,offset) , SEEK_SET) ;
        fread(buf , keySize , 1 , sstable) ;
    }
#endif
```

Figure 1: 模拟无缓存测试实现

测得实验数据：

KeyNum	Vlen\throughput(per ms)	Get
10	10	187.20
100	10	216.57
1000	10	47999.75
10000	10	62159.11
100000	10	168959.73

2. 内存中只缓存了SSTable的索引信息，通过二分查找从SSTable的索引中找到offset，并在磁盘中读取对应的值。

测试结果如下

KeyNum	Vlen\throughput(per ms)	Get
10	10	202.19
100	10	248.52
1000	10	2599.76
10000	10	2850.68
100000	10	3734.61

- 内存中缓存SSTable的Bloom Filter和索引，先通过Bloom Filter判断一个键值是否可能在一个SSTable中，如果存在再利用二分查找，否则直接查看下一个SSTable的索引

我在之前常规分析中的实现即为第三种（缓存BF），因此沿用先前的数据中Get的平均时延。可以看到完全在磁盘中寻找的性能远远落后于使用缓存的性能，尤其在需要硬盘查询的数据量下，硬盘查询的性能迅速变差。而不使用bf的性能略低于使用bf的，这可能是因为bf的误判率较高导致的。

2.1.4 Compaction的影响

测试实现方法：不断插入新数据，每隔0.5s统计前0.5s的吞吐量，连续测试30s，最终Level数为12层，得到测试结果如图。

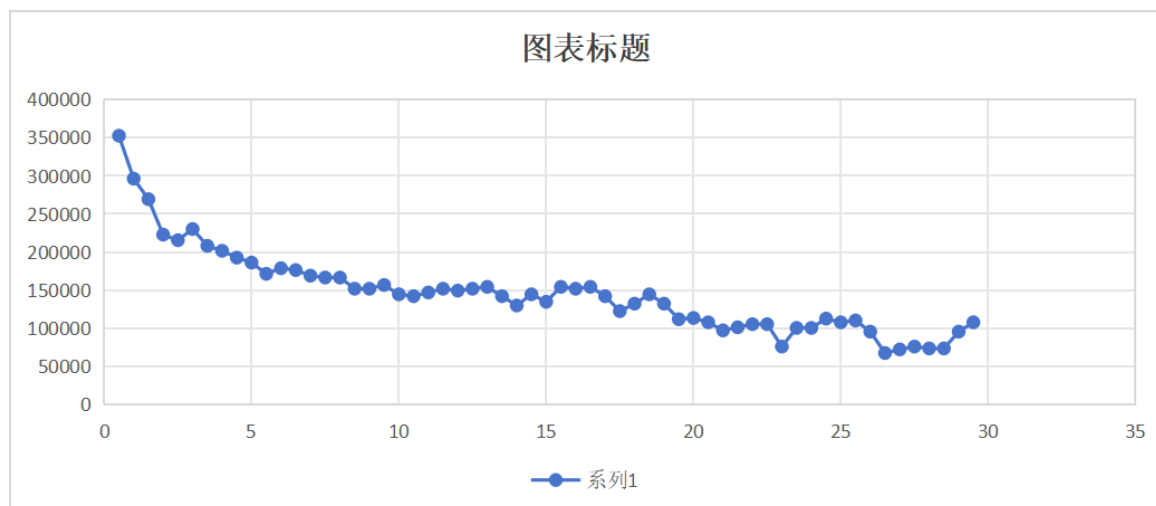


Figure 2: 吞吐量随时间的变化

测试刚开始5s时吞吐量迅速下降，这是因为前期Compaction层数小，发生次数快，每0.5s可能都有层数的增加。在5s后系统进入平稳状态，吞吐量变化呈阶梯状变化，在150000ops，100000ops，70000ops分别有一个阶梯，分别对应了一次Compaction导致的层数上升导致的写效率的下降。这三个阶梯可以看出Compaction在导致层数上升时，对性能的伤害。

2.1.5 Bloom Filter 大小配置的影响

Bloom Filter 大小的影响：Bloom Filter 过大会使得一个 SSTable 中索引数据较少，进而导致 SSTable 合并操作频繁；Bloom Filter 过小又会导致其 false positive 的几率过大，辅助查找的效果不好。

调整定义在头文件中的BF大小参数，并复用normalTest进行测试，以较有代表性的10000数据量为测试数据。先检验修改BF大小后的正确性，修改bf大小为4096byte, 2048byte ,9216byte, 7168byte , 12288byte , 14336byte进行测试，均通过正确性校验。使用normalTest测试文件进行测试结果如下：

BFSIZE/Byte	Put	Get
14436	1718.09	2734.12
12288	3794.88	3139.03
9216	2058.23	2820.81
8192	2203.14	2678.99
7168	6192.60	9380.44
4096	4665.68	3230.24
2048	1529.44	3182.83

对于Put，可以发现随着BF的大小走向最大/最小两端时，Put的时延逐渐减小。发现与预期的中间低两端高并不对应。

BF增大时延减小的原因，是一次合并需要遍历的键减小，参与合并的sstable键的范围与数量也减小了，因此Put的时延下降。

BF减小时延减小的原因，是因为sstable较大，需要的合并次数减少，因此Put的时延也下降了。

对于Get，可以发现Get在BF大小为7168附近时，时延较大，而在两端时延基本一致。此处的原因是BF适中，则键的数量与BF大小达到平衡点，误报率较高的同时，需要搜索的键也较多，因此时延有一个显著的提高。而对于两端来说，因为实现中搜索sstable时，先根据最大值与最小值查找找到可能的sstable，再通过bloomFilter，因此误报率较低的需要搜索的键就较多，误报率较高的需要在sstable中二分查找的键就较少，反而达到平衡效率，效率较均匀。

综上，与预期并不符合的原因，其实是因为误报率与键数量的配合，产生的时延是一个最大值和一个最小值，最小值在8192附近取得，而最小值在7168附近取得。

3 结论

对 Project 的整体总结，包括对实验结果的评价。本次Project完成度较高，实现了Compaction, GC, Persistence都完成的最终成果，并且从实验结果上来说，大部分的实验结果都符合预期，如无缓存时的较大时延，以及基本操作随KeyNum的增大而增大的结果。因此我认为此次项目实现成果良好，测试结果完整且较为全面，可以认为基本达成了项目与报告所要求达成的目标。

4 致谢

首先感谢同专业的各位同学们在交流中提供的宝贵实现经验与Debug经历，以及进行测试的经验，没有这些经验，完成项目并测试完成的时间可能会进一步的延后。

另外，感谢在其他软件工程专业课中学习到的面向对象设计思路的启发。使用该思路，我对工程进行了拆分，分为内存类，硬盘管理类与缓存类这三个大类，对项目调试起到了重要的帮助。

在实现跳表与BloomFilter的过程中，我从需要github上的开源工程与博客中的文章上获取了许多实现经验，对这些工程与文章的作者表示感谢与敬意。

5 其他和建议

这部分不做强制要求，你可以列出 LSM-KV Project中遇到的种种困难、挑战、bug、以及吐槽。

1. 困难 我个人认为最为困难的点在于内存与硬盘的管理，缓存与硬盘的一致性以及Debug上。内存管理要求每个地方必须谨慎地处理malloc的内存，不能出现泄露状况，否则将会导致整个系统的崩溃，而因为存在硬盘的反复读写，必须要小心地注意有没有重名文件，否则就会出现这些重名文件只是覆写的情况，而覆写之后，旧的文件应被删除，则会导致删除了新增的不该被删掉的文件，最终导致硬盘文件与真实插入文件的不一致。

Debug上来说，因为处理的都是二进制文件，无法当即查看是否写入硬盘正确，很难去判断是硬盘处理的错误，还是缓存中处理的错误，这点导致Debug十分困难

2. 吐槽 写完缓存才发现报告里要我不缓存，我今天真是爆炸了啊，做个测试到处改实现，还要担心会不会影响正确性。我自己的测试集设计的不够好，导致这个bf效果也很差，看不出很明显的优化。而且工作量好大，非常极限时间做完报告。